PDS PRACTICAL 1

NAME: DIVYA R. WAGHMODE

DIV: SY-A

ROLL NO.: A-28

Aim : Use sequence data types and their associated operations in Python

1. What are sequence data types in Python? Name them and explain how indexing and slicing work across different sequence types?

Sequence data types are data structures that store multiple elements in an ordered manner. Each element is assigned a position (index) which allows accessing elements individually.

Main Sequence Data Types in Python

1. **String (str):** A sequence of characters.
2. **List (list):** An ordered, mutable collection of elements.
3. **Tuple (tuple):** An ordered, immutable collection of elements.
4. **Range (range):** An immutable sequence of numbers, often used in loops.

Indexing

Indexing means accessing individual elements of a sequence using their position (index). In Python, indexing is zero-based, meaning the first element is at index 0.

```
text = "Python"
print(text[0])  # Output: P
print(text[3])  # Output: h
print(text[-1]) # Output: n (negative indexing accesses from the end)
```

Slicing

Slicing extracts a portion (a subsequence) from a sequence. It uses the syntax `sequence[start : end : step]`:

- `start`: The starting index of the slice (inclusive). If omitted, it defaults to the beginning of the sequence.
- `end`: The ending index of the slice (exclusive). If omitted, it defaults to the end of the sequence.

- `step`: The step size between elements (e.g., `2` for every other element). If omitted, it defaults to `1`.

my_list = [10, 20, 30, 40, 50, 60] print(my_list[1:5:2]) # Output: [20, 40] ```

2. Difference between List and Tuple in Python

Feature	List	Tuple
Mutability	Mutable (can change)	Immutable (cannot change)
Syntax	`[ ]`	`( )`
Performance	Slightly slower	Faster
Memory	Uses more memory	Uses less memory

Why prefer tuples in real-world applications?

Tuples are often preferred in scenarios where:

- **Data should not change:** When you have a collection of items that represent constant values or a fixed set of data that should not be modified during the program's execution, tuples are a suitable choice. This ensures data integrity.
- **Better performance and memory efficiency:** Due to their immutable nature, tuples are generally more memory-efficient and can be slightly faster to process than lists, especially for large datasets or when iterating through elements.
- **Used as dictionary keys:** Because tuples are immutable, they can be used as keys in dictionaries, which is not possible with mutable lists.
- **Safer for data integrity:** The immutability of tuples makes them inherently safer for data integrity, as once created, their contents cannot be accidentally or intentionally altered.

3. How Python arrays are different from lists

Python lists are general-purpose containers that can store items of different data types, while arrays (from the `array` module) are designed for efficient storage of homogeneous (same data type) numeric data.

Feature	List	Array
Data types	Can store mixed data types	Stores elements of the same data type
Performance	Slower for numeric operations	Faster for numeric operations
Memory	Uses more memory	More memory-efficient and compact
Flexibility	Highly flexible, dynamic size	Less flexible, fixed data type
Built-in	Built-in data type	Requires importing the `array` module

Example:

List

```
data = [1, 2, 'hello', 4.5]
print(data)
```

Array

```
import array
arr = array.array('i', [1, 2, 3, 4]) # 'i' specifies signed integer type
print(arr)
```

Here, `'i'` means it's an array designed to store signed integers.

When should you use arrays (from the `array` module)?

Use arrays when:

- **Storing large numeric datasets:** Arrays are more memory-efficient than lists for storing large collections of numbers.
- **Need better memory efficiency:** If memory usage is a critical concern, arrays can be a better choice.
- **Performing scientific or numerical computations:** While the built-in `array` module offers some advantages, for serious numerical computing, dedicated libraries like NumPy are generally preferred.

4. How dictionaries work internally in Python

A dictionary in Python is an unordered collection of data values, used to store data values like a map, which, unlike other data types that hold only a single value as an element, holds `key:value` pairs.

Example:

```
student = {
    "roll": 11,
    "name": "DIVYA",
    "grade": "A"
}

print(student["name"]) # Output: DIVYA
```

Internal Mechanism: Hash Tables

Internally, Python dictionaries are implemented using a **hash table**. This data structure allows for highly efficient storage and retrieval of key-value pairs.

How it works:

1. **Hashing the Key:** When you store a key-value pair, Python first applies a `hash()` function to the key. This function converts the key into an integer, called a hash value.
2. **Mapping to Memory Index:** The hash value then determines a specific memory location (or "bucket") within the hash table where the key-value pair will be stored.
3. **Storing the Pair:** The value is stored along with its corresponding key at that memory index.

Why dictionary lookup is fast

Dictionary access (lookup, insertion, deletion) is typically very fast, averaging **O(1) (constant time)** complexity. This means that as the number of items in the dictionary grows, the time it takes to find or modify an item remains roughly the same.

- **Direct Access:** The hash function directly points to the probable memory location of the key. There's no need to sequentially search through all elements.

Comparison of Search Times:

Data Structure	Search Time (Average)
List	O(n)
Tuple	O(n)
Dictionary	O(1)

This efficiency is why dictionaries are widely used for:

- **Database lookups:** Quickly retrieving records based on unique identifiers.
- **Caching:** Storing frequently accessed data for fast retrieval.
- **Data indexing:** Creating efficient mappings between data points.
- **Counting frequencies:** Tallying occurrences of items.

5. Difference between Sets and Lists in Python

Sets and Lists are both collection data types in Python, but they serve different purposes due to their distinct characteristics.

Feature	List	
Order	Ordered (maintains insertion order since Python 3.7)	Unordered (elements do not have a define
Duplicates	Allowed	Not allowed (stores only unique elements)
Indexing/Slicing	Supported (elements can be accessed by index)	Not supported (elements cannot be acces
Mutability	Mutable	Mutable (elements can be added or remov
Syntax	`[ ]`	`{ }` or `set()`
Performance	Slower for membership checking (`in` operator)	Faster for membership checking and remo

Example:

List

```
numbers_list = [1, 2, 2, 3, 4, 4, 5]
print("List with duplicates:", numbers_list)
# Output: List with duplicates: [1, 2, 2, 3, 4, 4, 5]
```

Set

```
numbers_set = {1, 2, 2, 3, 4, 4, 5}
print("Set (duplicates removed):", numbers_set)
# Output: Set (duplicates removed): {1, 2, 3, 4, 5} (order may vary)
```

How sets handle duplicates

Sets inherently store only unique elements. When you try to add a duplicate element to a set, Python processes it as follows:

1. Python first **hashes** the element. This hash value is used to determine where the element *would* be stored internally.
2. If an element with the same hash value and value already exists in the set, the new element is simply **ignored**. It's not added again, ensuring uniqueness.

Practical uses of sets

Sets are highly useful for various tasks, primarily due to their unique element property and fast operations:

- **Removing duplicates from data:** This is one of the most common uses. By converting a list or any iterable to a set and then back to a list, you can efficiently get a collection of unique items.
- **Fast membership testing:** Checking if an element exists in a set (`element in my_set`) is very efficient (average O(1) time complexity), making sets ideal for lookups where order is not important.
- **Mathematical set operations:** Sets provide methods for common mathematical set operations:
  - `union()`: Combines elements from two or more sets.
  - `intersection()`: Finds common elements between sets.
  - `difference()`: Finds elements in one set but not in another.
  - `symmetric_difference()`: Finds elements that are in either of two sets but not in both.
- **Checking for uniqueness:** Quickly determine if all elements in a collection are unique by comparing the length of the original collection to the length of its set

conversion.

```
print("STRING OPERATIONS")
text = "DataScience"
print("String:", text)
print("Length of string:", len(text))
print("First three characters:", text[:3])
```

```
STRING OPERATIONS
String: DataScience
Length of string: 11
First three characters: Dat
```

```
print("LIST MAX & MIN")
numbers = [12, 45, 7, 23, 56]
print("List:", numbers)
print("Maximum value:", max(numbers))
print("Minimum value:", min(numbers))
```

```
LIST MAX & MIN
List: [12, 45, 7, 23, 56]
Maximum value: 56
Minimum value: 7
```

```
print("TUPLE OPERATIONS")
students = ("Sarang", "Amit", "Rohit", "Neha", "Priya")
print("Student Names:")
for name in students:
    print(name)
```

```
TUPLE OPERATIONS
Student Names:
Sarang
Amit
Rohit
Neha
Priya
```

```
print("DICTIONARY OPERATIONS")
student_dict = {
    19: "Divya",
    22: "Amit",
    33: "Sahil",
    44: "Neha"
}

print("Dictionary:", student_dict)
print("Roll Numbers:")
for key in student_dict.keys():
    print(key)
```

```
DICTIONARY OPERATIONS
Dictionary: {19: 'Divya', 22: 'Amit', 33: 'Sahil', 44: 'Neha'}
Roll Numbers:
19
22
33
44
```

```
print("SET OPERATIONS")
list_numbers = [1,2,3,4,2,3,5,6,9,5,1]
print("Original List:", list_numbers)

unique_set = set(list_numbers)
print("Set (Duplicates removed):", unique_set)
```

```
SET OPERATIONS
Original List: [1, 2, 3, 4, 2, 3, 5, 6, 9, 5, 1]
Set (Duplicates removed): {1, 2, 3, 4, 5, 6, 9}
```

```
print("VOWEL COUNT")
string = input("Enter a string: ")

vowels = "aeiouAEIOU"
count = 0

for char in string:
    if char in vowels:
        count += 1

print("Number of vowels:", count)
```

```
VOWEL COUNT
Enter a string: PYTHON IS EASY PROGRAMMING LANGUAGE
Number of vowels: 11
```

```
print("LIST INSERTION & DELETION")
num_list = [10,20,30,40,50]
print("Original List:", num_list)
```

```
num_list.insert(2,25)
print("After insertion:", num_list)

num_list.remove(30)
print("After deletion:", num_list)
```

```
LIST INSERTION & DELETION
Original List: [10, 20, 30, 40, 50]
After insertion: [10, 20, 25, 30, 40, 50]
After deletion: [10, 20, 25, 40, 50]
```

```
print("TYPE CONVERSION")
list1 = [1,2,3,4]
tuple1 = tuple(list1)
print("List to Tuple:", tuple1)

tuple2 = (5,6,7,8)
list2 = list(tuple2)
print("Tuple to List:", list2)
```

```
TYPE CONVERSION
List to Tuple: (1, 2, 3, 4)
Tuple to List: [5, 6, 7, 8]
```

```
print("Range Example")
print("Numbers using range:")
for i in range(1,6):
    print(i)
```

```
Range Example
Numbers using range:
1
2
3
4
5
```

Conclusion:

Thus, the practical on sequence data types and their associated operations in Python was successfully performed. Through this practical, the use of different sequence data types such as strings, lists, tuples, ranges, dictionaries, and sets was understood clearly. Operations like indexing, slicing, insertion, deletion, type conversion, duplicate removal, and iteration were implemented successfully. This practical helped in understanding how Python handles ordered collections of data efficiently and improved the basic programming skills required for data manipulation and problem-solving.